

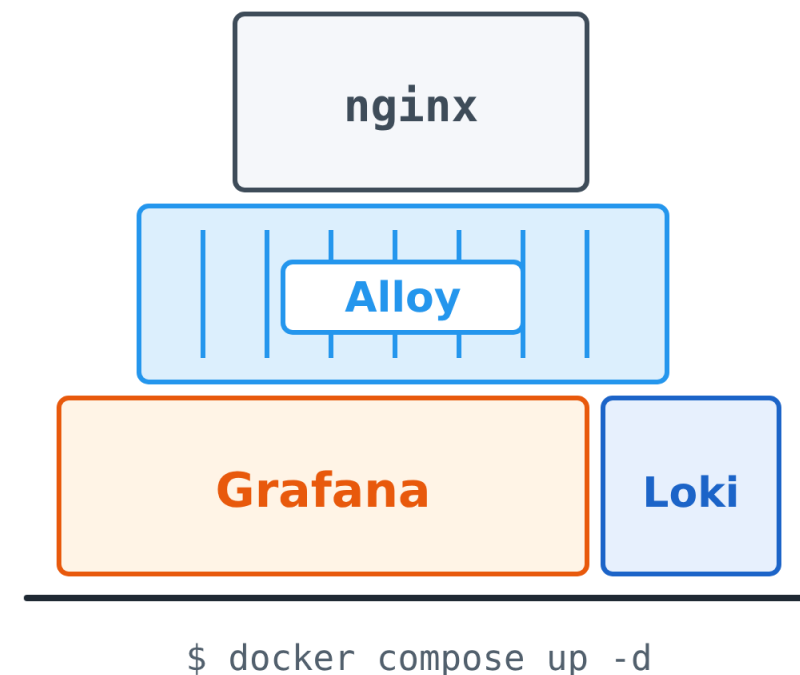
LEZIONE 3

Docker e Docker Compose

ITS CyberSecurity - Jesi (AN) 2026/2027

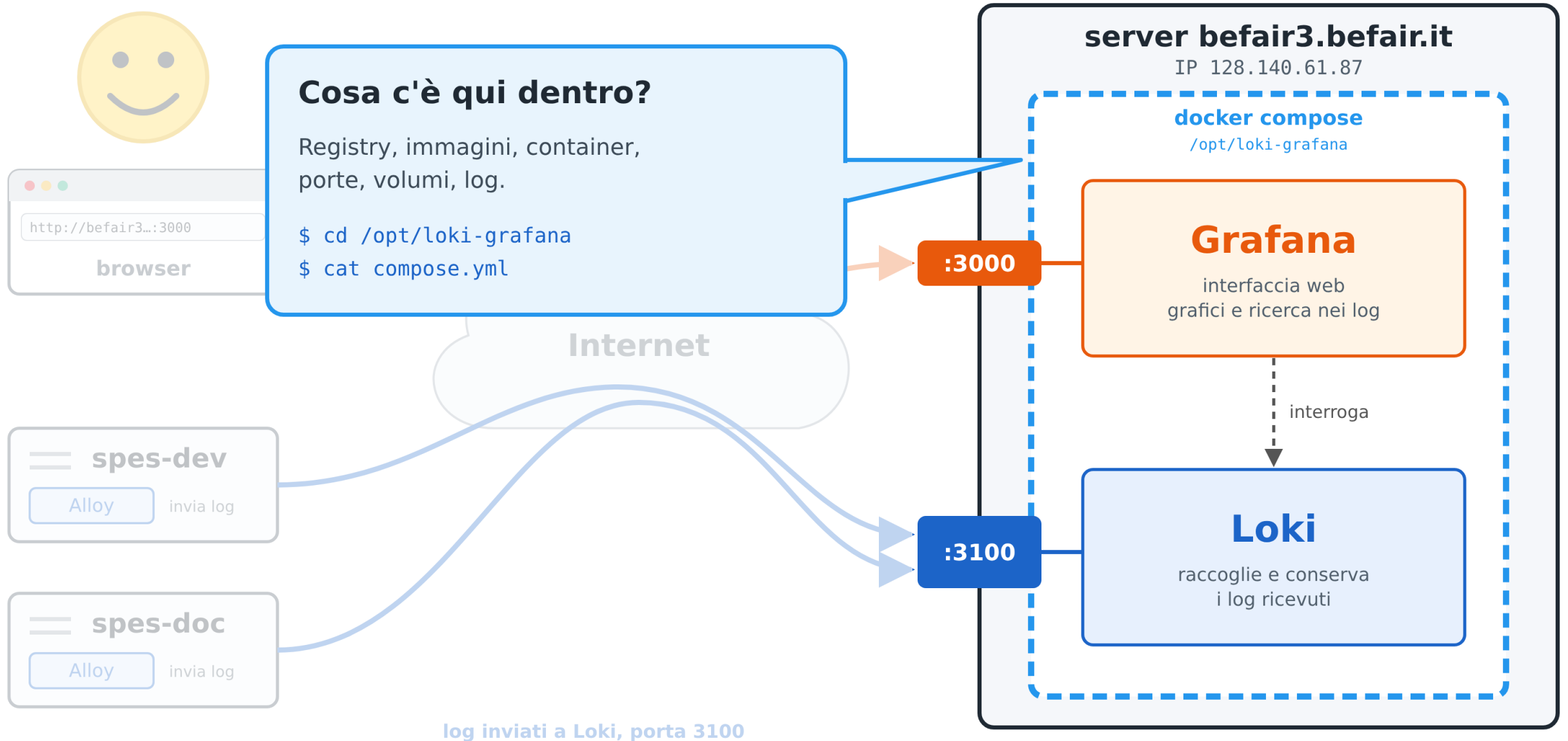
Corso Logging, Monitoring, Troubleshooting

prof. Luca Ferroni



Dove eravamo: la scatola chiusa

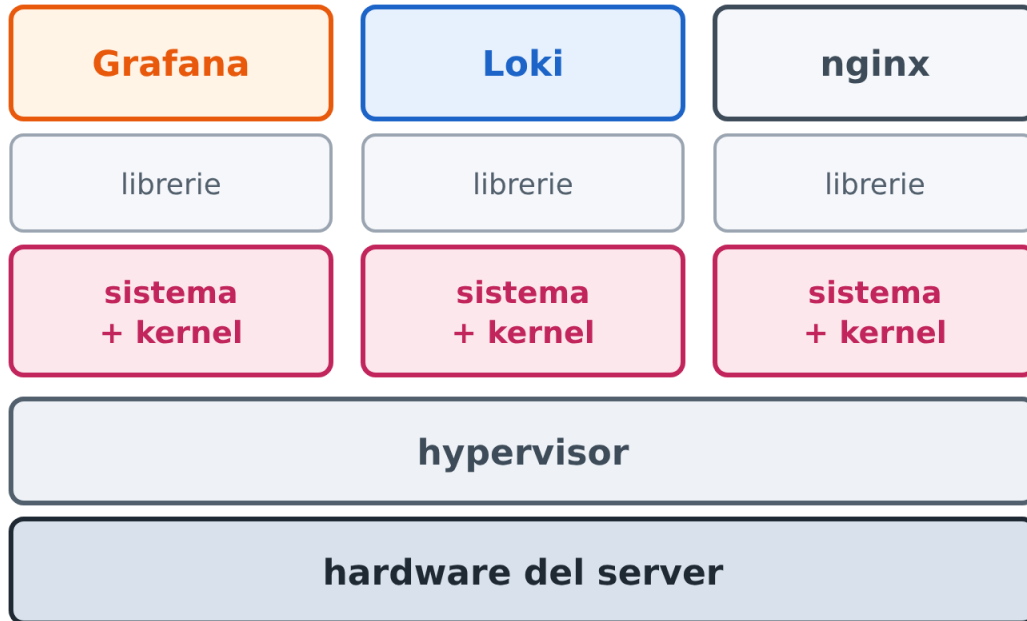
Nella lezione 1 abbiamo usato docker compose senza aprirlo. Oggi lo apriamo.



Un container non è una macchina virtuale

Ricordi la lezione 2? Kernel, demoni, applicazioni. Il container condivide il kernel dell'host.

Macchine virtuali



Container

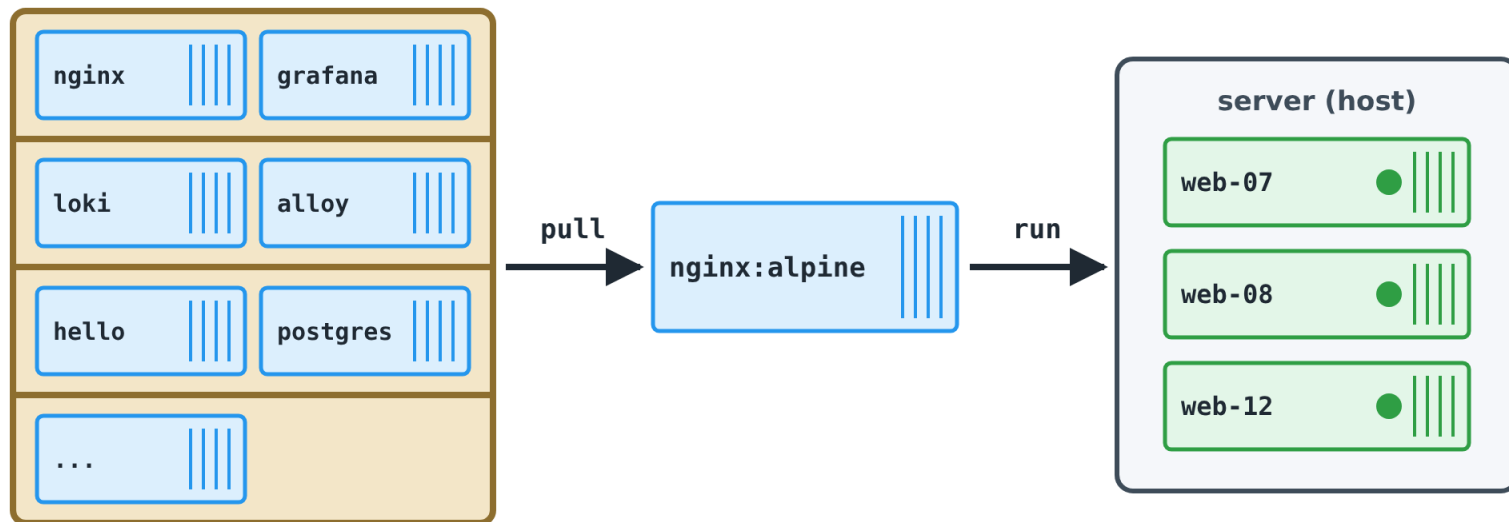


Cosa cambia in pratica

- Un container parte in un secondo: non deve avviare un sistema operativo.
- Ognuno porta con sé le sue librerie: "sul mio PC funziona" vale anche sul server.
- Un container Linux ha bisogno di un kernel Linux: su Windows e Mac Docker Desktop usa una VM nascosta.
- Un processo nel container è un processo dell'host: lo vedi con ps aux, come il miner della lezione 2.

Registry, immagini, container

Le tre parole da sapere (ridisegnato da Microsoft Learn, "Docker overview")



Registry

il magazzino: conserva tante immagini con nome e versione

Immagine

un pacchetto fermo, in sola lettura: programma, librerie, configurazione

Container

un'immagine in moto: un processo vivo, con le sue porte e i suoi log

Una sola immagine, tanti container

Come una classe e i suoi oggetti: da `nginx:alpine` ognuno di voi avvia il suo.
Se cancello un container l'immagine resta; se cancello l'immagine, al prossimo run Docker la scarica dal registry.

Quali registry esistono

Pubblici, su Internet

- Docker Hub: quello predefinito
- GitHub (ghcr.io)
- Quay (quay.io)
- i registry di AWS, Google, Azure

Privati, in azienda

- GitLab, Harbor, Nexus
- un registry installato in casa: anche lui è... un container

Come si legge un nome

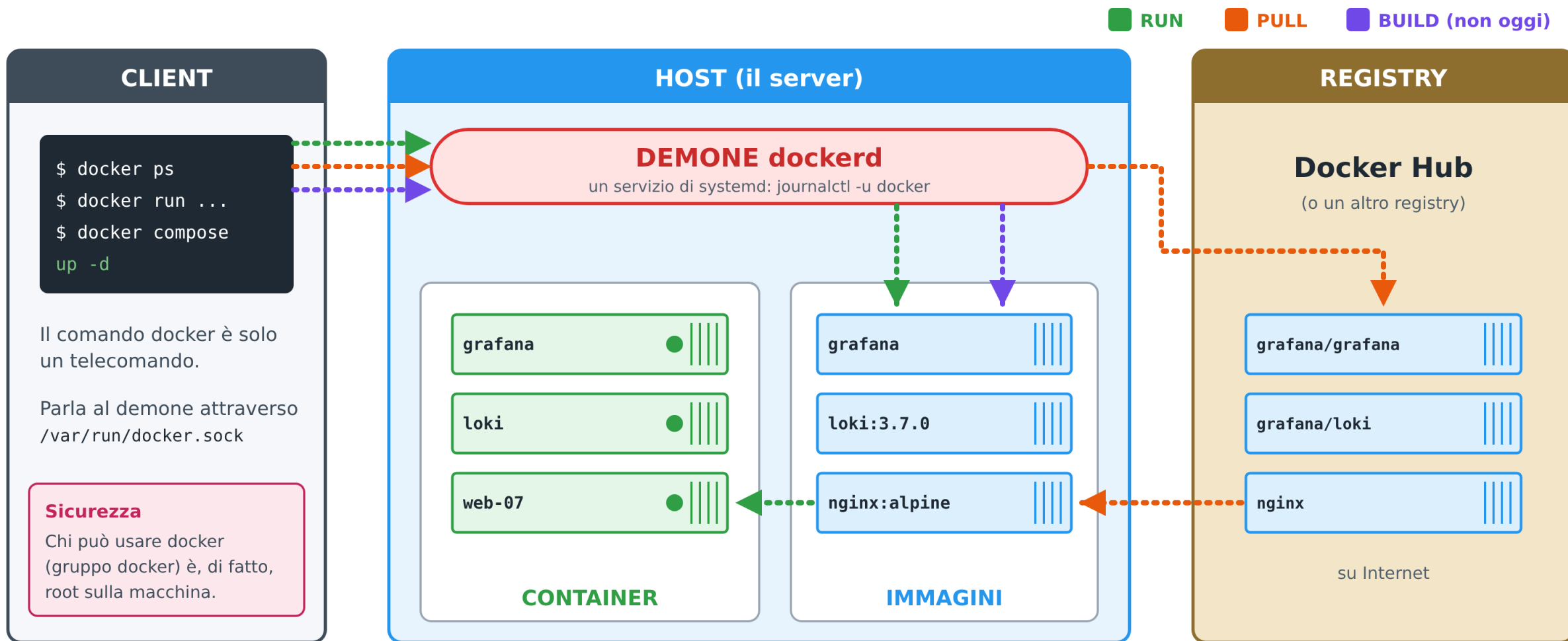
`grafana/loki:3.7.0`

grafana = chi la pubblica
loki = il nome dell'immagine
3.7.0 = la versione (il "tag")

Senza tag si prende `:latest`, che cambia nel tempo.

Chi fa cosa: client, demone, registry

Il comando docker non fa niente da solo: chiede al demone (ridisegnato da Nordic APIs, via devopedia.org)



docker pull nginx:alpine

← scarica l'immagine dal registry

docker run -d -p 8007:80 nginx:alpine

← la scarica se manca, poi avvia il container

Torniamo al centro commerciale

Le parole di Docker, tradotte nella metafora della lezione 2

DOCKER	NEL CENTRO COMMERCIALE
registry (Docker Hub)	il catalogo del franchising da cui si ordinano i progetti dei negozi
immagine	il progetto standard di un negozio uguale per tutti, esiste solo sulla carta
container	un negozio aperto, costruito dal progetto dallo stesso progetto se ne aprono tanti
-p 8007:80	la porta 8007 sulla facciata del centro porta al bancone interno numero 80
dockerd	la direzione del centro commerciale apre, chiude e controlla i negozi
docker logs	il registro di cassa del negozio tutto ciò che il negozio ha detto e fatto

Il centro commerciale (indirizzo IP) è il server. Dentro, ogni negozio ha il suo bancone 80; in facciata, porte diverse: 8007, 8008...

Il primo container: hello-world

Leggiamo l'output: Docker ci spiega da solo l'architettura della slide precedente

```
$ docker run hello-world
```

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
Status: Downloaded newer image for hello-world:latest
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

To generate this message, Docker took the following steps:

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal.

Prima volta: pull

Le prime 3 righe compaiono solo se l'immagine manca.

1

Il client parla al demone

il telecomando, attraverso docker.sock

2

Il demone scarica dal registry

pull: l'immagine non c'era sul server

3

Crea il container e lo avvia

run: l'immagine diventa un processo

4

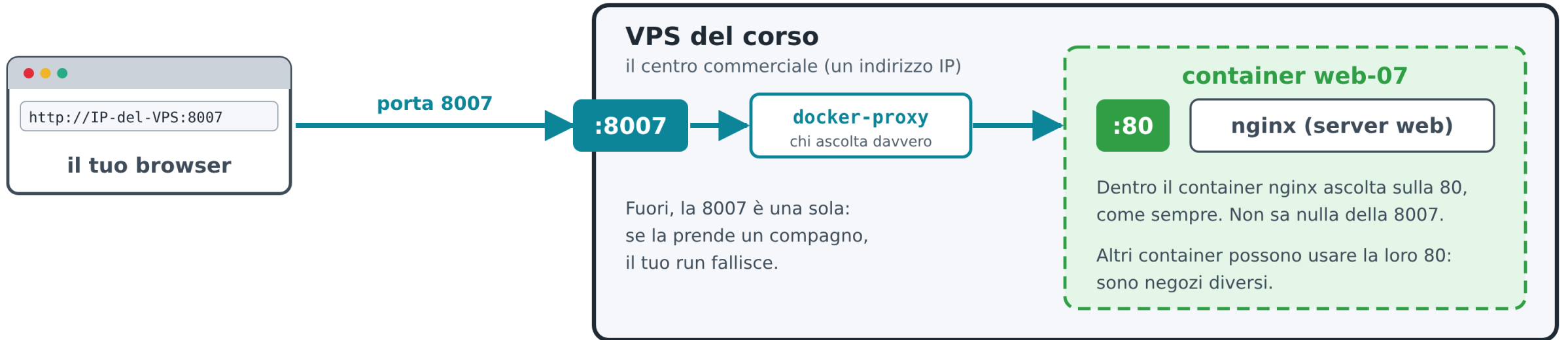
L'output torna al terminale

è il log del container: docker logs

Poi: **docker ps** è vuoto, **docker ps -a** mostra Exited (0). Un container vive quanto il suo processo.

Un negozio per ciascuno: nginx e le porte

-p PORTA_ESTERNA:PORTA_INTERNA. Ognuno usa la porta 8000 + il suo numero (qui il numero 7)



```
$ docker run -d --name web-07 -p 8007:80 nginx:alpine
3f9cle0a7b2d... # -d: in background, torna subito il prompt

$ docker ps
CONTAINER ID   IMAGE          STATUS          PORTS                               NAMES
3f9cle0a7b2d   nginx:alpine   Up 5 seconds    0.0.0.0:8007->80/tcp               web-07

$ curl -I http://localhost:8007
HTTP/1.1 200 OK
Server: nginx

$ docker run -d --name web-07b -p 8007:80 nginx:alpine
Error ... Bind for 0.0.0.0:8007 failed: port is already allocated
```

Attenzione: è su Internet

Una porta pubblicata con -p è aperta a tutto il mondo, anche se il firewall ufw dice di no: Docker scrive le sue regole prima.

Solo dal server stesso:

```
-p 127.0.0.1:8007:80
```

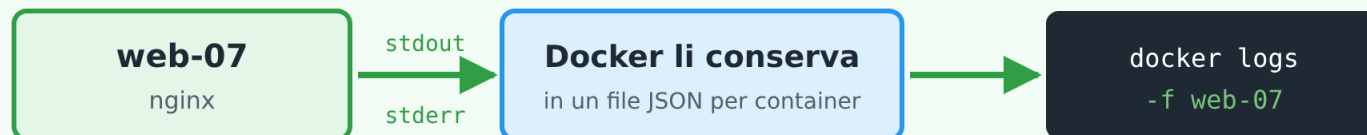
Fine lezione: pulizia.

```
docker rm -f web-07
```

Dove finiscono i log

Due strade diverse: i log dei container e i log del demone Docker

I log di un container



nginx nel container non scrive in `/var/log/nginx`: scrive sullo schermo.
Docker raccoglie quell'output. Il file sta in
`/var/lib/docker/containers/<id>/<id>-json.log`
ma non si legge a mano. Con compose:

`docker compose logs -f grafana` (come nella lezione 1)

I log del demone



`dockerd` è un servizio di `systemd`,
come `sshd`: i suoi log vanno nel
journal (lezione 2). Qui si vede
quando Docker si è fermato o
riavviato, e perché.

```
$ docker logs -f web-07
203.0.113.25 - - [15/Oct/2026:09:12:44 +0000] "GET / HTTP/1.1" 200 615 "-" "Mozilla/5.0 (X11; Linux x86_64)" "-"
203.0.113.25 - - [15/Oct/2026:09:12:51 +0000] "GET /pippo HTTP/1.1" 404 153 "-" "curl/8.5.0" "-"

$ docker logs web-07 2>/dev/null | awk '{print $1}' | sort | uniq -c
    14 203.0.113.25
     3 198.51.100.7

# quasi il formato di apache_access.log (in più l'ultimo campo): valgono i comandi della lezione 2
```

Occhio all'ora: i container di solito sono in UTC (+0000). Come nella lezione 1: 03:30 UTC = 05:30 in Italia.

Opzioni utili: `--since 1h`, `--tail 50`, `-t` (aggiunge il timestamp di Docker a ogni riga)

Il sito non risponde: un livello alla volta

Lo stesso metodo delle lezioni 1 e 2, con un gradino in più: il container

DAL CLIENT · PORTA

nc

Il negozio è aperto?

```
$ nc -zv IP-del-VPS 8007
nc: connect to ... port 8007 (tcp)
failed: Connection refused
```

refused: il server c'è, ma sulla 8007 non ascolta nessuno.
Si passa sul server.

SUL SERVER · CHI ASCOLTA

SS

C'è qualcuno sulla porta?

```
$ sudo ss -ltnp | grep 8007
# nessuna riga: nessuno ascolta
# se gira: users(("docker-proxy",...))
```

Con Docker sulla porta non vedi nginx ma docker-proxy.
Nessuna riga: il container è giù?

SUL SERVER · DOCKER

docker ps -a

Il container gira?

```
$ docker ps -a
NAMES      STATUS
web-07     Exited (0) 2 minutes ago
```

Senza -a vedi solo quelli accesi.
Exited: si è fermato. Da solo o qualcuno l'ha fermato?

SUL SERVER · LOG

docker logs

Cosa ha detto il container?

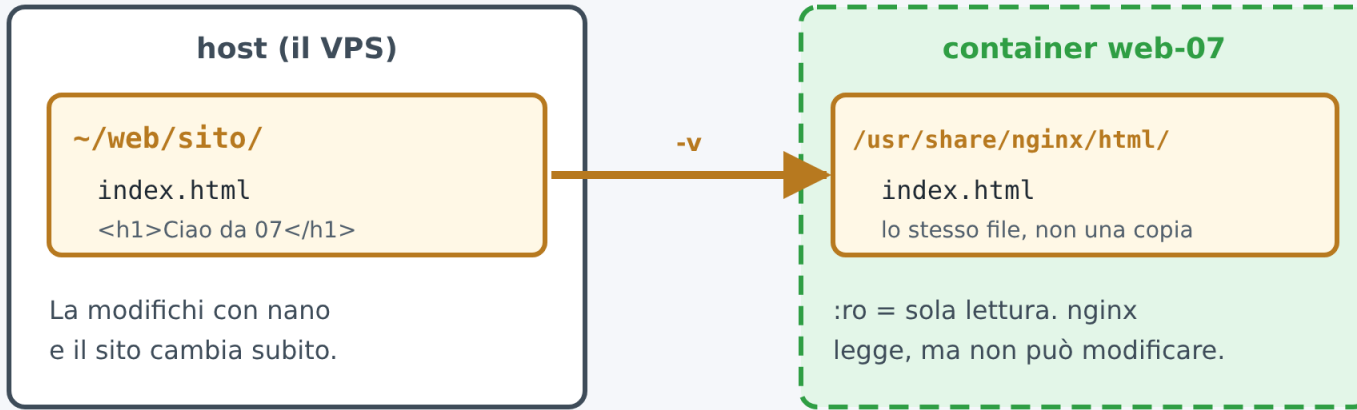
```
$ docker logs --tail 20 web-07
... signal 3 (SIGQUIT) received,
    shutting down
```

Qualcuno l'ha spento (docker stop).
Poi: docker start web-07
e si ricomincia dal primo livello.

Volumi: i dati stanno fuori dal container

Il container si butta e si ricrea. Quello che deve restare (pagine, configurazioni, dati) va montato da fuori.

Una cartella dell'host dentro il container



`-v ~/web/sito:/usr/share/nginx/html:ro` HOST : CONTAINER : opzioni

Lo vedrete nei nostri compose

Loki (server)

`./loki/config.yml:/etc/loki/config.yml:ro`
la configurazione, scritta da noi
`loki-data:/loki`
un volume con nome: i log ricevuti sopravvivono se il container muore

Alloy (client, lezione 4)

`/var/log/journal:/var/log/journal:ro`
Alloy legge il journal dell'host (lezione 2) senza poterlo toccare

```
$ mkdir -p ~/web/sito && echo "<h1>Ciao da 07</h1>" > ~/web/sito/index.html
$ docker rm -f web-07
$ docker run -d --name web-07 -p 8007:80 -v ~/web/sito:/usr/share/nginx/html:ro nginx:alpine

# e ora entriamo nel container, come con ssh in un altro server
$ docker exec -it web-07 sh
/ # ls /usr/share/nginx/html
index.html
/ # cat /etc/os-release      # Alpine Linux, non la distribuzione dell'host
/ # exit
```

Da docker run a compose.yml

Lo stesso container, scritto in un file invece che in una riga lunghissima

Prima: un comando da ricordare

```
$ docker run -d --name web-07 -p 8007:80 -v ~/web/sito:/usr/share/nginx/html:ro nginx:alpine
```

Dopo: ~/web/compose.yml

```
services:
  web:
    image: nginx:alpine
    ports:
      - "8007:80"
    volumes:
      - ./sito:/usr/share/nginx/html:ro
    restart: unless-stopped
```

YAML: l'indentazione conta, si usano spazi e mai tab.

Il file si legge, si versiona con git, si copia.

- **web** è il nome del servizio. Il container si chiamerà web-web-1: cartella-servizio-numero.
- **image**: quale progetto di negozio usare.
- **ports**: come -p, ESTERNA:INTERNA.
- **volumes**: come -v. I percorsi con ./ partono dalla cartella del file (~/web).
- **restart**: riparti da solo se ti fermi o se il server si riavvia. Ci torneremo.

Un compose.yml può avere più servizi: Loki e Grafana stanno nello stesso file e si trovano per nome (<http://loki:3100>).

I comandi minimi di docker compose

Sempre dalla cartella che contiene compose.yml

`docker compose up -d`

crea e avvia tutti i servizi del file

-d: in background. Se cambi il file, rilancialo: ricrea solo ciò che è cambiato

`docker compose ps`

chi gira e chi no, con le porte

come docker ps, ma solo per questo progetto

`docker compose logs -f grafana`

i log di un servizio, in tempo reale

senza nome: tutti i servizi insieme. Ctrl+C per uscire

`docker compose stop / start`

ferma e riavvia, senza cancellare

i container restano, li vedi con ps -a

`docker compose restart loki`

ferma e riparte un servizio

utile dopo aver cambiato un file di configurazione

`docker compose down`

ferma e CANCELLA i container

i volumi con nome restano; con -v cancella anche quelli (e i dati!)

`docker compose pull`

scarica le versioni nuove delle immagini

poi up -d per usarle

Lezione 1: cd /opt/loki-grafana, poi ps, logs grafana, up -d. Erano già questi comandi.

Riapriamo il caso della lezione 1

Perché Grafana non era ripartito da solo? Leggiamo il compose.yml di befair3

```
/opt/loki-grafana/compose.yml
```

```
services:
```

```
  loki:
```

```
    image: grafana/loki:3.7.0
```

```
    command: -config.file=/etc/loki/config.yml
```

```
    volumes: ...
```

```
    ports:
```

```
      - "3100:3100"
```

```
    # restart: ???
```

```
  grafana:
```

```
    image: grafana/grafana:latest
```

```
    ports:
```

```
      - "3000:3000"
```

```
    volumes: ...
```

```
    depends_on:
```

```
      - loki
```

```
    # restart: ???
```

03:30 UTC

Grafana e Loki si spengono

nei log: Shutting down (docker compose logs)

dopo

Nessuno li riaccende

senza restart restano Exited. Chi li ha spenti? journalctl -u docker

mattina

Il browser: servizio irraggiungibile

è l'inizio della lezione 1

in aula

docker compose up -d

ripartono a mano. Funziona, ma è una toppa

La cura: una riga per servizio

restart: unless-stopped

Il compose di Alloy ce l'ha già.

Le politiche di restart

no (predefinita) non riparte mai

on-failure solo se esce con un errore

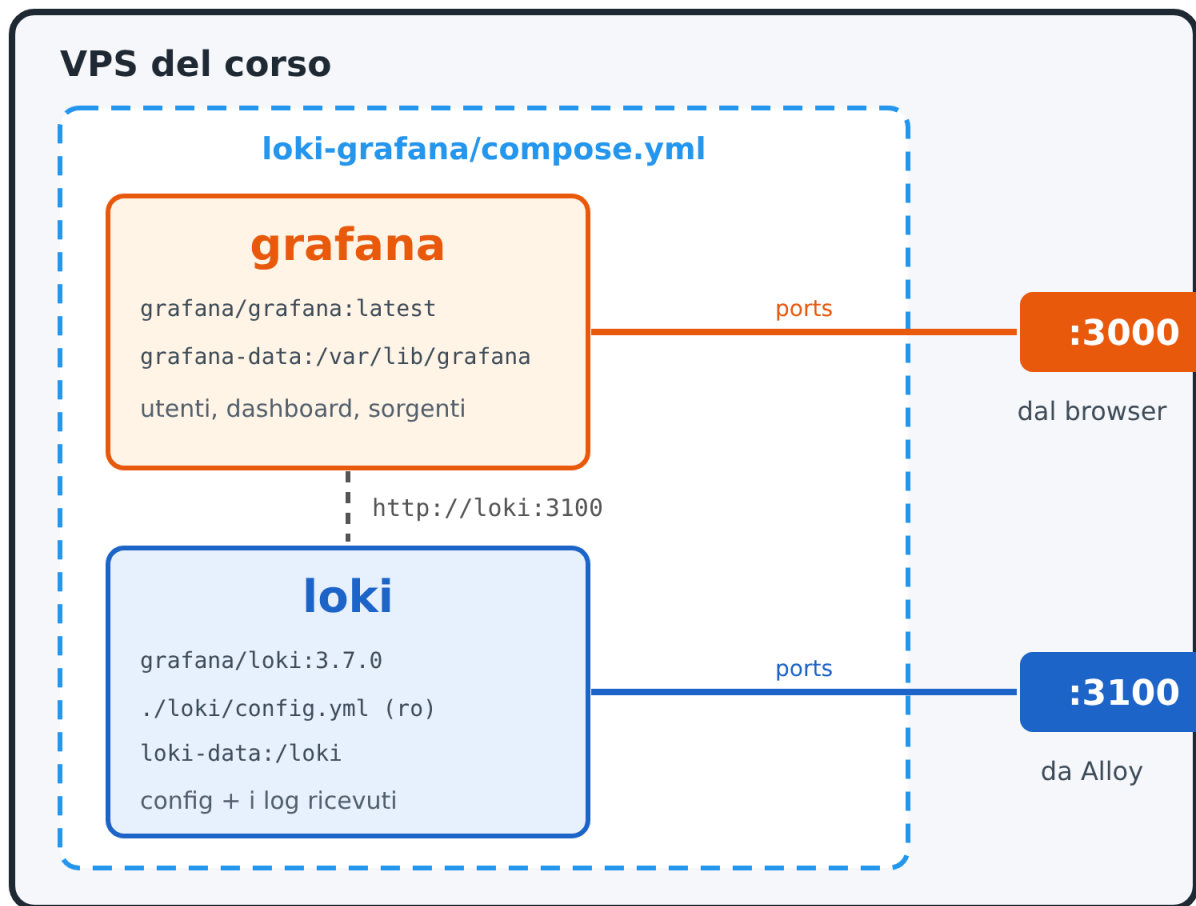
always sempre, anche se l'avevi fermato tu

unless-stopped sempre, tranne se l'hai fermato tu

Prova in aula: con i container accesi, sudo systemctl restart docker. Prima senza restart, poi con unless-stopped.

Da befair3 al VPS del corso

Lo stesso compose.yml, su un altro server: questo è il bello dei container



Lo facciamo insieme

- 1 **cd its-jesi-lessons/logging/loki-grafana**
il compose è nel repository del corso
- 2 **docker compose up -d**
scarica le immagini e avvia Loki e Grafana
- 3 **docker compose ps**
entrambi Up? Se no: docker compose logs loki
- 4 **curl http://IP-del-VPS:3100/ready**
dal vostro PC. Risposta attesa: ready
- 5 **http://IP-del-VPS:3000**
browser: admin / admin, poi nuova password
- 6 **Data sources > Loki > http://loki:3100**
dentro il compose Loki si chiama per nome

Lezione 4: Alloy sui vostri PC (logging/journal-alloy)

In alloy/config.alloy si cambiano due righe: url (l'indirizzo del Loki del VPS) e host (il nome del vostro PC).

Compiti: verificate se sul vostro PC gira `docker run hello-world` (Linux, WSL2 o Docker Desktop).

Prima di uscire

Rispondete in poche righe

- 1 Che differenza c'è tra un'immagine e un container?
- 2 Dove leggo i log di un container?
- 3 E quelli del demone Docker?
- 4 Cosa fa restart: unless-stopped, e perché sarebbe servito nella lezione 1?

Per la prossima lezione

- Sul vostro PC: docker run hello-world funziona? Linux, WSL2 o Docker Desktop: annotate quale.
- Leggete logging/journal-alloy/compose.yml: quali volumi monta, e perché in sola lettura?
- Lezione 4: Alloy manda i vostri log al VPS.

I comandi di oggi

<code>docker run -d --name N -p E:I img</code>	avvia un container
<code>docker ps / docker ps -a</code>	accesi / tutti
<code>docker logs -f N</code>	log del container
<code>docker exec -it N sh</code>	entra nel container
<code>docker rm -f N</code>	ferma e cancella
<code>docker images</code>	immagini scaricate
<code>docker compose up -d</code>	avvia il progetto
<code>docker compose ps</code>	stato del progetto
<code>docker compose logs -f S</code>	log di un servizio
<code>docker compose down</code>	ferma e cancella
<code>journalctl -u docker</code>	log del demone